

# Lab Assignment 6:

## Recursive vs. Iterative

DAEN 300 - Fall 2026

Fri Oct 2 2026

### Overview

A music streaming company collects the number of times listeners play music from different artists. Your task is to create a small ranking system that sorts artists from the highest number of plays to the lowest.

You will create two implementations:

1. an **iterative** implementation using loops;
2. a **recursive** implementation using a function that calls itself.

You will then compare the two approaches using:

- correctness;
- Big-O time complexity;
- measured execution time using Python's `timeit` module;
- readability and implementation style.

### Learning Objectives

By the end of this lab, you should be able to:

- store and process a small collection of data in Python;
- sort and rank records using a numeric value;
- implement an algorithm using iteration;
- implement an algorithm using recursion;
- identify approximate Big-O time complexity;
- measure execution time using `timeit`;
- compare theoretical complexity with actual execution time.

# Scenario

You are a junior data engineer working for a fictional music streaming company called **DataBeats**. The company has collected the following play counts for ten artists:

```
music_data = [  
    ("Taylor Swift", 950),  
    ("Bad Bunny", 870),  
    ("Billie Eilish", 620),  
    ("Drake", 810),  
    ("SZA", 590),  
    ("The Weeknd", 780),  
    ("Olivia Rodrigo", 540),  
    ("Bruno Mars", 690),  
    ("Kendrick Lamar", 730),  
    ("Sabrina Carpenter", 660)  
]
```

Each tuple has the form

(artist name, play count)

Your goal is to rank the artists so that the artist with the largest play count receives rank 1.

## Part 1: Explore the Data

Create the `music_data` list in Python.

Write Python code that prints every artist and the corresponding number of plays.

For example:

|               |           |
|---------------|-----------|
| Taylor Swift  | 950 plays |
| Bad Bunny     | 870 plays |
| Billie Eilish | 620 plays |

Answer the following questions:

1. How many artist records are in the dataset?
2. Which field should be used to create the ranking?
3. Should the ranking use ascending or descending order?

## Deliverable 1

Submit your Python code and your answers to the three questions.

## Part 2: Iterative Ranking

Create a function called:

```
def rank_iterative(data):  
    pass
```

The function must rank the artists from highest play count to lowest play count.

For this part of the lab, do **not** use:

```
sorted()
```

or

```
list.sort()
```

Instead, write a simple sorting procedure using loops and comparisons.

One possible strategy is to repeatedly compare neighboring records and swap them when they are in the wrong order.

Your function may begin with:

```
def rank_iterative(data):  
    result = data.copy()  
  
    # Your loop-based sorting algorithm  
  
    return result
```

The final ranking should resemble the following:

| Rank | Artist       | Play Count |
|------|--------------|------------|
| 1    | Taylor Swift | 950        |
| 2    | Bad Bunny    | 870        |
| 3    | Drake        | 810        |
| 4    | The Weeknd   | 780        |
| ...  | ...          | ...        |

Answer the following:

1. How many loops does your implementation use?
2. Does the number of comparisons increase when the dataset becomes larger?
3. Approximately how does the number of comparisons grow as the number of artists increases?

## Deliverable 2

Submit:

- your `rank_iterative()` function;
- the ranked output;
- short answers to the three questions.

## Part 3: Recursive Ranking

Create another ranking function called:

```
def rank_recursive(data):  
    pass
```

This version must use **recursion**.

A recursive function solves a problem by calling itself using a smaller version of the original problem.

A recursive algorithm usually requires two components:

Recursive Algorithm = Base Case + Recursive Case

The **base case** determines when the recursion stops.

A possible structure is:

```
def rank_recursive(data):  
    # Base case  
  
    # Perform comparisons  
  
    # Recursive call  
  
    return data
```

Test the function using the same ten artists.

The iterative and recursive implementations should produce the same result.

You may verify this using:

```
iterative_result = rank_iterative(music_data)  
recursive_result = rank_recursive(music_data)  
  
print(iterative_result == recursive_result)
```

The expected output is:

True

Answer the following:

1. What is the base case in your recursive implementation?
2. What changes during each recursive call?
3. What would happen if the base case were missing?

## Deliverable 3

Submit:

- your `rank_recursive()` function;

- the ranked result;
- the result of the equality test;
- your answers to the three questions.

## Part 4: Big-O Analysis

Big-O notation describes how the amount of work performed by an algorithm grows as the input size increases.

Let

$n$  = number of artists.

Common complexity classes include:

$$O(1)$$

$$O(\log n)$$

$$O(n)$$

$$O(n \log n)$$

$$O(n^2)$$

A simple sorting algorithm that repeatedly compares elements may require work that grows approximately as

$$T(n) \propto n^2.$$

For example:

| Number of Artists $n$ | $n^2$     |
|-----------------------|-----------|
| 10                    | 100       |
| 20                    | 400       |
| 50                    | 2,500     |
| 100                   | 10,000    |
| 1,000                 | 1,000,000 |

Complete the following table for your implementations.

| Implementation | Main Technique  | Expected Big-O | Explanation |
|----------------|-----------------|----------------|-------------|
| Iterative      | Loops           |                |             |
| Recursive      | Recursive calls |                |             |

Answer the following questions:

1. What is the approximate Big-O time complexity of your iterative solution?
2. What is the approximate Big-O time complexity of your recursive solution?
3. Does using recursion automatically make an algorithm faster?
4. If two algorithms have the same Big-O complexity, must they have exactly the same execution time?

Explain each answer in one or two sentences.

## Deliverable 4

Submit the completed table and your Big-O analysis.

## Part 5: Measure Execution Time with `timeit`

Big-O describes theoretical growth. We can also measure the actual execution time of Python code.

Python provides the `timeit` module for this purpose.

Begin with:

```
import timeit
```

Use code similar to the following:

```
iterations = 1000

iterative_time = timeit.timeit(
    lambda: rank_iterative(music_data),
    number=iterations
)

recursive_time = timeit.timeit(
    lambda: rank_recursive(music_data),
    number=iterations
)

print("Iterative:", iterative_time)
print("Recursive:", recursive_time)
```

Calculate the average execution time per call:

$$t_{\text{average}} = \frac{t_{\text{total}}}{\text{number of executions}}$$

For example:

```
iterative_average = iterative_time / iterations
recursive_average = recursive_time / iterations
```

Complete the following table.

| Implementation | Runs | Total Time (s) | Average Time per Run |
|----------------|------|----------------|----------------------|
| Iterative      | 1000 |                |                      |
| Recursive      | 1000 |                |                      |

Your execution times may differ from those of other students. Runtime depends on the computer, operating system, Python version, and other programs running at the same time.

Answer:

Which implementation was faster on your computer, and by approximately how much?

## Deliverable 5

Submit:

- your `timeit` code;
- your completed timing table;
- your answer to the performance question.

## Part 6: Compare the Two Implementations

Complete the following table.

| Question                      | Iterative | Recursive |
|-------------------------------|-----------|-----------|
| Produces the correct ranking? |           |           |
| Uses loops?                   |           |           |
| Uses recursion?               |           |           |
| Big-O time complexity         |           |           |
| Measured average runtime      |           |           |
| Easier for you to understand? |           |           |

Write a short conclusion of approximately 4–6 sentences.

Your conclusion should address:

1. Which implementation was faster in your experiment?
2. Did both implementations have the same Big-O complexity?
3. Why can two implementations with the same Big-O complexity have different measured runtimes?
4. Which implementation would you choose for this small dataset, and why?

There is not necessarily one correct answer to the final question. Support your decision using your implementation and measurements.

## Deliverable 6

Submit your completed comparison table and conclusion.

## Python's Built-in Sorting

If you finish early, create a third implementation using Python's built-in sorting function:

```
def rank_python(data):  
    return sorted(  
        data,  
        key=lambda artist: artist[1],  
        reverse=True  
    )
```

Measure its execution time using `timeit`.

Then complete the following table:

| Implementation               | Big-O         | Average Runtime |
|------------------------------|---------------|-----------------|
| Iterative                    |               |                 |
| Recursive                    |               |                 |
| Python <code>sorted()</code> | $O(n \log n)$ |                 |

Answer:

Why would a professional data engineer normally prefer Python's built-in sorting functionality instead of writing a sorting algorithm manually?

## Submission

Submit one Jupyter Notebook (`.ipynb`) or Python file (`.py`) containing:

| Item | Deliverable                                                     |
|------|-----------------------------------------------------------------|
| 1    | Music dataset and initial exploration                           |
| 2    | Iterative ranking implementation                                |
| 3    | Recursive ranking implementation                                |
| 4    | Verification that both implementations produce the same ranking |
| 5    | Big-O analysis table                                            |
| 6    | <code>timeit</code> experiment and timing table                 |
| 7    | Iterative-versus-recursive comparison table                     |
| 8    | 4–6 sentence conclusion                                         |

---

## Tasks

---

Explore the music dataset  
Build the iterative ranking function  
Build the recursive ranking function  
Analyze Big-O complexity  
Run `timeit` experiments  
Complete the comparison table  
Write conclusion and check submission

---

## Final Question

Suppose DataBeats grows from 10 artists to 10,000 artists.  
Would you still use the sorting algorithm you created in this lab?  
Consider the relationship among:

|                                                                   |
|-------------------------------------------------------------------|
| Correctness + Complexity + Measured Performance + Code Simplicity |
|-------------------------------------------------------------------|

A data engineer should consider all four when selecting an implementation!!